Assignment 1 — Branch Prediction

Al-Akhir Nayan

PSID: 2371302

Pentium M direction predictor

Programming assignment report

Overview

For this assignment I built the direction-prediction part of the Pentium M branch predictor: a bimodal table combined with a tagged global table. With four global ways it averages 5.848 mispredictions per 1,000 instructions (MPKI) over the 20 traces that come with the infrastructure. Below I describe how it works, compare it with a few other configurations, and discuss what the numbers show.

AI use: I used an AI assistant (Claude) to help with the code and this report, as permitted for this assignment.

Assignment 1: Pentium M branch predictor

How to build and run

```
cd src
make -B CXXFLAGS='-O3 -Wall'   # builds the predictor on macOS
cd ..
csh ./run traces        # MPKI for every trace and the average
./src/predict traces/164.gzip/gzip.trace.bz2      # a single trace
```

- I built the predictor on macOS with `make -B CXXFLAGS='-O3 -Wall'`, which leaves out the debug symbols. All commands are run from `cbp2-infrastructure-v2/`.
- `predict_extra_credit` (the complete Pentium M predictor) is still the empty placeholder from the starter code. All results in this report come from `pm_predictor`.
- `run` is a csh script. If it isn't executable, start it with `csh ./run traces`, or run `chmod +x run` first.
- The trace reader passes the file name to bzip2 without quotes, so a path containing spaces breaks it. Using relative paths, as above, avoids the problem.

Build options (add them to CXXFLAGS in src/Makefile):

flag	effect
-DPM_GLOBAL_WAYS=2	2-way global predictor instead of 4-way
-DPM_ALLOCATE_ALWAYS	allocate a global entry on every global miss, not only when the bimodal predictor was wrong

1. What I implemented

My code is the `pm_predictor` class in `src/my_predictor.h`. It covers the two parts the assignment requires, the bimodal table and the global predictor (the blue blocks in the diagram). The loop predictor, target prediction (BTB/iBTB) and the PIR are optional, and I left them out.

lookup	input / index	result
Bimodal	IP[11:0]	Prediction from a two-bit counter
Global	HASH = (IP XOR GHR)[14:0]; set = HASH[14:6]; tag = HASH[5:0]	Search four tagged ways
Selection	Global tag hit?	Use global on a hit; otherwise use bimodal

structure	size	storage
Bimodal table	4096 two-bit counters, index = IP[11:0]	8 Kbit
Global predictor	512 sets × 4 ways; each entry: valid, 6-bit tag, 2-bit counter, LRU	18 Kbit (+ LRU)
Global history register	15 bits	

Prediction (predict), conditional branches only:

1. Read the bimodal counter at IP[11:0].

2. Form HASH = (IP XOR GHR)[14:0], then search the 4 ways of set HASH[14:6] for tag HASH[5:0].

3. On a tag hit, the final prediction is the global counter; otherwise it is the bimodal counter.

Unconditional branches are predicted taken. Target prediction returns 0 (not required).

Update (update), conditional branches only:

1. The bimodal counter is always trained (saturating ±1).
2. On a global hit, the global counter is trained and the way becomes most recently used.
3. On a global miss, a new global entry is allocated only if the bimodal prediction was wrong. It takes an invalid way if there is one, otherwise the LRU way, and its counter starts weakly toward the actual outcome. This follows the published description of the Pentium M: only branches the bimodal table gets wrong use space in the global table.
4. The outcome is shifted into the GHR.

The real Pentium M hashes the IP with a path information register (PIR). Since the PIR is optional here, I used a global history register of conditional-branch outcomes instead, the same GHR-XOR-address idea as the example in Appendix B of the assignment.

I wrote the global table and its LRU replacement myself inside the predictor class, rather than using an existing cache implementation.

2. Results

All numbers are direction mispredictions per 1,000 instructions (MPKI). Each trace is 100 million instructions, so MPKI is simply the number of misses divided by 100,000. Averages are taken over the unrounded values for all 20 traces. Columns marked A allocate a global entry on every global miss, instead of only when the bimodal prediction was wrong.

trace	always taken	bimodal only	gshare 32K (sample)	PM 2-way	PM 2-way A	PM 4-way	PM 4-way A
gzip	70.025	15.030	12.473	12.520	12.617	**12.436**	12.459
vpr	85.312	14.671	13.415	13.866	14.337	**13.298**	14.070
gcc	86.071	19.347	11.254	10.618	15.298	**9.285**	13.304
mcf	102.497	33.245	15.837	16.398	17.680	**15.621**	16.488
crafty	48.651	13.075	5.837	5.983	8.464	**5.211**	6.895
parser	78.421	15.417	10.008	9.036	10.357	**8.538**	9.242
compress	76.983	9.356	7.831	7.854	8.199	**7.702**	7.839
jess	92.343	8.809	1.562	1.149	1.750	**1.118**	1.464
raytrace	109.506	3.806	2.756	2.503	3.421	**2.307**	2.952
db	91.451	6.480	3.909	3.723	4.613	**3.615**	4.283
javac	109.644	3.181	2.267	2.126	2.638	**2.037**	2.397
mpegaudio	110.339	3.154	2.188	2.124	2.604	**2.028**	2.352
mtrt	112.022	3.706	2.657	2.470	3.268	**2.383**	2.957
jack	77.958	7.691	3.033	2.420	3.946	**2.353**	3.075
eon	24.226	9.405	1.807	1.501	3.139	**1.425**	1.837
perlbmk	66.833	5.955	2.554	1.917	4.700	**1.819**	3.279
gap	75.950	9.675	3.926	3.604	4.797	**3.507**	3.814
vortex	70.524	1.916	1.222	0.854	1.239	**0.828**	1.085
bzip2	178.257	0.112	0.094	0.059	0.076	**0.057**	0.073
twolf	59.221	21.942	21.489	21.981	22.046	**21.389**	22.236
average	86.312	10.299	6.306	6.135	7.260	**5.848**	6.605

I used four global ways for the main configuration. Here is part of the output saved in `results_pm4.txt`:

```
traces/164.gzip/gzip.trace.bz2          12.436
traces/175.vpr/vpr.trace.bz2            13.298
traces/176.gcc/gcc.trace.bz2            9.285
...
traces/300.twolf/twolf.trace.bz2        21.389
average MPKI: 5.847
```

The run script cuts its average off at three decimals, which is why it prints 5.847. The actual average is 5.8478665, so the table shows 5.848. In the same way, the PM 2-way A average of 7.2596035 rounds to 7.260.

3. Discussion

- Compared with the bimodal table alone (10.299 MPKI), the four-way hybrid brings the average down to 5.848, about 43% fewer mispredictions. It also beats the sample gshare predictor (6.306 MPKI) by about 7%. I didn't test the tagged global table on its own, so I can't say how much of the gain it would give by itself.
- For storage, the bimodal counters take 8 Kbit and the four-way global table another 18 Kbit for valid bits, tags and counters, not counting history and LRU state. (The C++ version uses more memory than that, since each counter is a whole byte and each entry has an integer LRU stamp.) The tags keep different branches apart within a set, but two PC/history combinations can still hash to the same value.
- Going from two ways to four lowers the average from 6.135 to 5.848. The biggest gains are on gcc (10.618 to 9.285) and crafty (5.983 to 5.211). Both versions have 512 sets, though, so the four-way table also has twice as many entries; part of the improvement comes from the extra capacity, not just the higher associativity.
- The allocation policy made a clear difference. With four ways, allocating on every global miss raises the average from 5.848 to 6.605. My guess is that allocating only after a bimodal miss keeps the table free for the branches that actually need it, but I didn't count evictions, so I haven't confirmed this.
- twolf, mcf, vpr and gzip stay above 12 MPKI with every predictor I tried. The totals alone don't explain why; separating the effects of capacity, aliasing, loop behaviour or path history would take more experiments.

Source: pm_predictor (src/my_predictor.h)

```
//
// Pentium M hybrid branch predictors
// This class implements a simple hybrid branch predictor based on the Pentium M branch outcome prediction units.
// Instead of implementing the complete Pentium M branch outcome predictors, the class below implements a hybrid
// predictor that combines a bimodal predictor and a global predictor.
//
// Structure (see the Pentium M diagram in the assignment):
//   - Bimodal table: 4096 2-bit counters, index = IP[11:0].
//   - Global predictor: 512 sets x PM_GLOBAL_WAYS ways. Each entry has a
//     valid bit, a 6-bit tag, a 2-bit counter and an LRU stamp.
//     HASH = IP[14:0] XOR GHR[14:0]; index = HASH[14:6], tag = HASH[5:0].
//     (The real Pentium M hashes the IP with a path information register;
//     the assignment lets us skip the PIR, so a global history register of
//     conditional-branch outcomes is used instead.)
//   - Final prediction: global prediction on a tag hit, bimodal otherwise.
//
// Update policy:
//   - The bimodal counter is always trained.
//   - On a global hit the global counter is trained and the way becomes MRU.
//   - On a global miss a new entry is allocated (LRU victim) only when the
//     bimodal predictor was wrong, so the global table holds only branches
//     the bimodal table cannot handle. Define PM_ALLOCATE_ALWAYS to allocate
//     on every miss instead (the policy used in the Appendix B example).

#ifndef PM_GLOBAL_WAYS
#define PM_GLOBAL_WAYS      4   // the assignment also allows 2
#endif
#define PM_BIMODAL_BITS     12  // 4096 entries
#define PM_GLOBAL_INDEX_BITS    9   // 512 sets
#define PM_GLOBAL_TAG_BITS  6
#define PM_HASH_BITS        (PM_GLOBAL_INDEX_BITS + PM_GLOBAL_TAG_BITS) // 15

struct pm_global_entry {
   bool valid;
   unsigned char tag;
   unsigned char ctr;      // 2-bit saturating counter
   unsigned int lru;       // larger = more recently used
};

class pm_update : public branch_update {
public:
       unsigned int index;     // bimodal index
       unsigned int set;       // global set
       unsigned int tag;       // global tag
       int way;            // hit way, -1 on a global miss
       bool bimodal_prediction;
};

class pm_predictor : public branch_predictor {
public:
       pm_update u;
       branch_info bi;
       unsigned int history;
       unsigned int lru_clock;
       unsigned char bimodal[1<<PM_BIMODAL_BITS];
       pm_global_entry global[1<<PM_GLOBAL_INDEX_BITS][PM_GLOBAL_WAYS];

       pm_predictor (void) : history(0), lru_clock(0) {
       // counters start weakly not-taken; the global table starts empty
       memset (bimodal, 1, sizeof (bimodal));
       memset (global, 0, sizeof (global));
       }

       branch_update *predict (branch_info & b) {
       bi = b;
       if (b.br_flags & BR_CONDITIONAL) {
          // bimodal lookup
          u.index = b.address & ((1<<PM_BIMODAL_BITS)-1);
          u.bimodal_prediction = bimodal[u.index] >> 1;

          // global lookup
```

```
        unsigned int hash = (b.address ^ history) & ((1<<PM_HASH_BITS)-1);
        u.set = hash >> PM_GLOBAL_TAG_BITS;
        u.tag = hash & ((1<<PM_GLOBAL_TAG_BITS)-1);
        u.way = -1;
        for (int w = 0; w < PM_GLOBAL_WAYS; w++) {
          pm_global_entry &e = global[u.set][w];
          if (e.valid && e.tag == u.tag) {
            u.way = w;
            break;
          }
        }

        // global hit overrides the bimodal prediction
        if (u.way >= 0)
          u.direction_prediction (global[u.set][u.way].ctr >> 1);
        else
          u.direction_prediction (u.bimodal_prediction);
      } else {
        u.direction_prediction (true);
      }
      // branch target prediction is not required
      u.target_prediction (0);
      return &u;
      }

      void update (branch_update *bu, bool taken, unsigned int target) {
      if (!(bi.br_flags & BR_CONDITIONAL)) return;
      pm_update *p = (pm_update *) bu;

      // bimodal table is always trained
      train (bimodal[p->index], taken);

      if (p->way >= 0) {
        pm_global_entry &e = global[p->set][p->way];
        train (e.ctr, taken);
        e.lru = ++lru_clock;
      } else {
#ifndef PM_ALLOCATE_ALWAYS
        if (p->bimodal_prediction != taken)
#endif
          allocate (p->set, p->tag, taken);
      }

      history = ((history << 1) | taken) & ((1<<PM_HASH_BITS)-1);
      }

      // replace an invalid way if there is one, otherwise the LRU way
      void allocate (unsigned int set, unsigned int tag, bool taken) {
      int victim = 0;
      for (int w = 0; w < PM_GLOBAL_WAYS; w++) {
        if (!global[set][w].valid) { victim = w; break; }
        if (global[set][w].lru < global[set][victim].lru) victim = w;
      }
      pm_global_entry &e = global[set][victim];
      e.valid = true;
      e.tag = tag;
      e.ctr = taken ? 2 : 1;      // weakly toward the actual outcome
      e.lru = ++lru_clock;
      }

      static void train (unsigned char &c, bool taken) {
      if (taken) {
        if (c < 3) c++;
      } else {
        if (c > 0) c--;
      }
      }
};
```